

## 摘要

本作品完成了一个“无源”交流电流表及无线读表器，整机不含电池与外部供电接口，全部能量取自被测线路。电流互感器采用单组带抽头次级绕组，同时承担取能与电流传感：次级经 SS16 肖特基全桥整流后向储能电容充电，再由低压差 LDO 建立 3.4 V 电源。为避免两条回路相互牵制，系统按时间片工作——空闲期专用于取电储能，测量窗口由互补 GPIO 控制 PMOS、NMOS 接入传感前端，这一分时复用是本设计的主要创新点。控制器以片内运放构成可编程增益放大级，由 DAC 提供偏置，将双极性信号搬移至 ADC 输入区间；固件逐帧扣除直流均值后算真有效值，按幅度自动换档，并由实测标定表插值修正。读数经 OLED 显示并由 BLE 上传，手机端完成远程读取、越限报警、记录与趋势显示。实测 0.119~2.136 A 内最大相对误差 0.20%，满足题目 0.5% 的精度要求，方案可行。

**关键词：**无源电流表；MSPM0G3507；真有效值；分时取能；低功耗；BLE

## 一、 方案比较与论证

### 1. 取能与传感方案

方案一：两组独立次级绕组，分别驱动取电电路和传感电路。两条通道互相隔离，但两组绕组同时占用磁芯窗口，绕制量大，匝数和耦合差异会引入一致性误差。

方案二：一组带抽头的次级绕组，用 MOS 开关对取电和传感分时复用。空闲期断开传感前端，互感器输出全部经全桥向储能电容充电；测量时接入传感前端，系统改由储能电容供电。

采用方案二。差别在冷启动：线路电流小时取能功率也小，常驻负载会拖慢储能电压建立，分时复用则让传感前端在充电阶段完全不取电。次级 270 匝并设抽头，实测后以 170 匝接入整流桥。

### 2. 整流与稳压电源方案

三个方案的前段相同：次级经全桥整流后向储能电容充电，齐纳二极管  $D_Z$  限制储能节点电压——线路电流达 2.0 A 时次级开路电压远高于器件耐压，钳位不可省略。区别只在储能节点之后得到工作电压的方式，电路见附录图 2。

方案一：不设稳压器，钳位点即工作电压。器件最少、没有压差损耗，但这条轨在充放电之间一直移动，齐纳的钳位点本身又随着电流和温度漂，MCU、OLED 和蓝牙全挂上面。

方案二：钳位点取在略高于 3.6 V 处，再由低压差、低噪声 LDO (TPS7A4701) 稳到 3.4 V。压差为 0.2 V，线性稳压的损耗可以忽略；得到一条不随储能电压摆动的轨，降低整流纹波和 MOS 开关的扰动。代价是多一级器件及其静态电流。

方案三：改用降压型开关变换器 (BUCK 电路)，搬运的是能量而不是电荷，效率不受压差牵制，但是冷启动时间长；启动死区会把小电流吞掉；需引入功率电感、过压钳位，复杂度高。

采用方案二。三者中只有它既给出稳定的轨、又不在冷启动阶段设门槛：LDO 没有最小启动电压，输入建立到哪里输出就跟到哪里；压差压到 0.2 V 后，相对 BUCK 的效率劣势也基本消失。

### 3. 电流有效值测量与量程方案

方案一：外部偏置与外部运放。电路独立、调试直观，但要增加运放、基准和增益切换器件，占板面积并带来额外静态功耗。

方案二：用 MSPM0G3507 片内 OPA 和 DAC。DAC 产生偏置，OPA 完成放

大，1~32 倍增益由软件配置并按采样幅值切换。

采用方案二，并用 ADC 采样序列直接算真有效值，而不是整流滤波测平均值再折算——后者只对正弦成立，而标定要对标的是标准表的真有效值读数。

## 二、 理论分析与计算

### 1. 电路关键参数计算

忽略励磁电流与损耗时，互感器原、副边安匝平衡，

$$N_1 I_1 = N_2 I_2, \quad (1)$$

故副边电流为

$$I_2 = \frac{N_1}{N_2} I_1. \quad (2)$$

取  $N_1 = 5$ 、接入匝数  $N_2 = 170$ ，原边 2.0 A 对应副边 58.8 mA。副边匝数越多，低电流下的感应电压越高，绕组电阻和分布参数也越大。

桥式整流每半周有两只二极管串联导通。设次级瞬时电压为  $u_2(t)$ 、单只二极管正向压降为  $V_F$ ，则整流后瞬时电压为

$$u_r(t) = \max(|u_2(t)| - 2V_F, 0). \quad (3)$$

输入电压越低， $2V_F$  占比越大，故全桥和隔离二极管一律选肖特基。

储能电容要独立支撑一次测量。设测量窗口内储能电压由  $V_H$  降到  $V_L$ ，线性稳压交付负载的是电荷，故容量下限为

$$V_o C_s (V_H - V_L) \geq E_{\text{measure}}, \quad (4)$$

其中  $V_o = 3.4 \text{ V}$ ， $E_{\text{measure}}$  为一次约 260 ms 窗口的能耗， $V_L$  不得低于 3.4 V 加 LDO 压差。压差越低  $V_L$  越低，同一只电容能交出的电荷越多，这是选低压差器件的直接原因。

## 三、 电路与程序设计

### 1. 硬件电路设计

系统框图如图 1 所示。

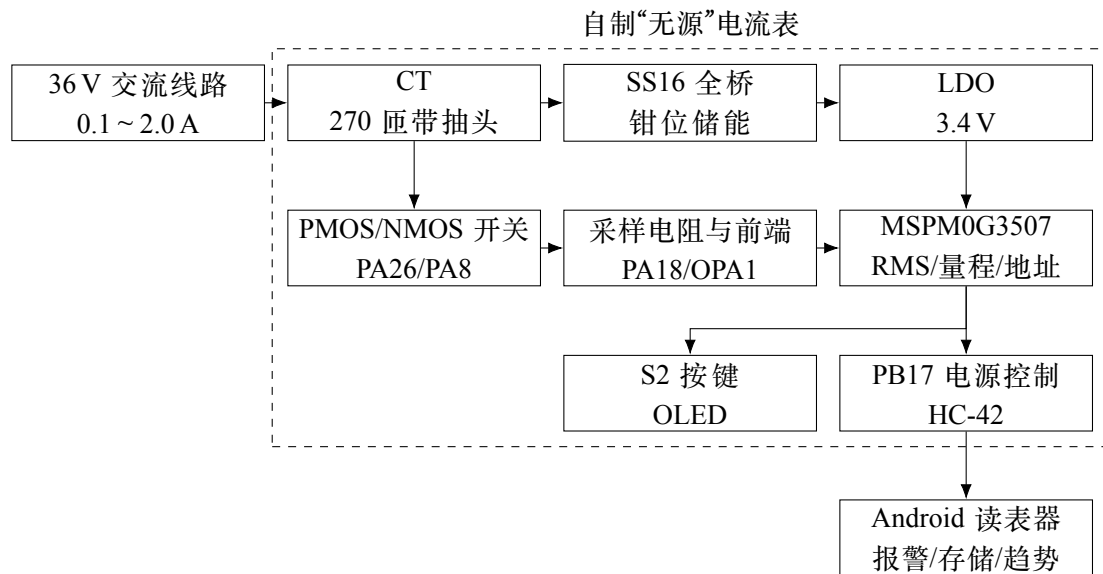


图 1 系统框图

整流输出记为节点  $A$ ， $A$  与储能电容之间串一只肖特基  $D_z$ ：采集支路接入时把  $A$  拉低，若无  $D_z$ ，冷启动积攒的能量会经采集支路反向放掉。储能电容并接一只串限流电阻的按钮，按下即放净残余电量。

采集支路自  $A$  取出，由 PMOS 高边和 NMOS 低边双端截断，不给取能功率留旁路，采样电阻  $R_s$  串在两者之间，其与 PMOS 漏极的交点送 PA18。PA26 经  $10\text{ k}\Omega$  上拉至  $A$ 、PA8 经  $10\text{ k}\Omega$  下拉至地，使上电到 MCU 接管 GPIO 之前支路保持断开。

完整原理图见附录图 3，图中同名网络（ $A$ 、 $3.4\text{ V}$ 、GND 以及各引脚名）相连。

模拟前端全部在片内，见附录图 4。PA18 上的采样电压经 P-MUX 进 OPA1 同相输入端，反相端接片内电阻梯的抽头，抽头位置即 GAIN 档，对应 2、4、8、16、32 倍五级增益。梯底不接地而接 DAC12 输出，放大作用在输入与 DAC 之差上，DAC 因此只移动放大的支点、不改变增益：固件按探测帧测得的直流反解 DAC 码，把输出重新摆回 ADC 中点。ADC1 用两个通道，CH1 取 OPA1 输出，CH3 直接取不经放大的直采档，用于输入直流偏低、任何增益档都无法居中的情形，也是探测帧本身所走的通道。ADC 与 DAC 共用片内  $2.5\text{ V}$  基准而不是供电电压，使供电纹波和 LDO 输出误差不进入读数。

## 2. 软件设计

固件以 embedded Rust 编写，no\_std 裸机运行，不链接标准库与操作系统，

异步执行器用 embassy。附录代码 1 与 2 分别是标定表的编译期校验和主循环对两路触发的等待。

程序上电停下来等电源：每 200 ms 用 ADC0 读一次  $V_{DD}$ ，不到 3.3 V 就不执行程序。基准和 ADC0 读完即掉电。

每次转换之前须先等基准建立 20 ms：VREF 的 READY 位表示基准内核已运行，电压到位后不等请求先完成一次测量，结束后把 PB17 置高、等待 350 ms 并初始化 UART2。

此后主循环等待三件事：S2 下降沿、蓝牙收到 MEAS、2 s 心跳到期。前两者走同一条测量路径；心跳只发一帧保活。测量期间到达的请求一律丢弃而不排队，测量结束时清空串口积压字节。

一次测量的时序如图 2：切换为传感态并等 20 ms 建立，采 160 点/40 ms 探测帧定增益与 DAC 偏置，再采 800 点/200 ms 正式帧；然后按 TIMG6/ADC1、OPA1/DAC12、基准、外部 MOS 开关的顺序关断，最后才计算有效值并查表。

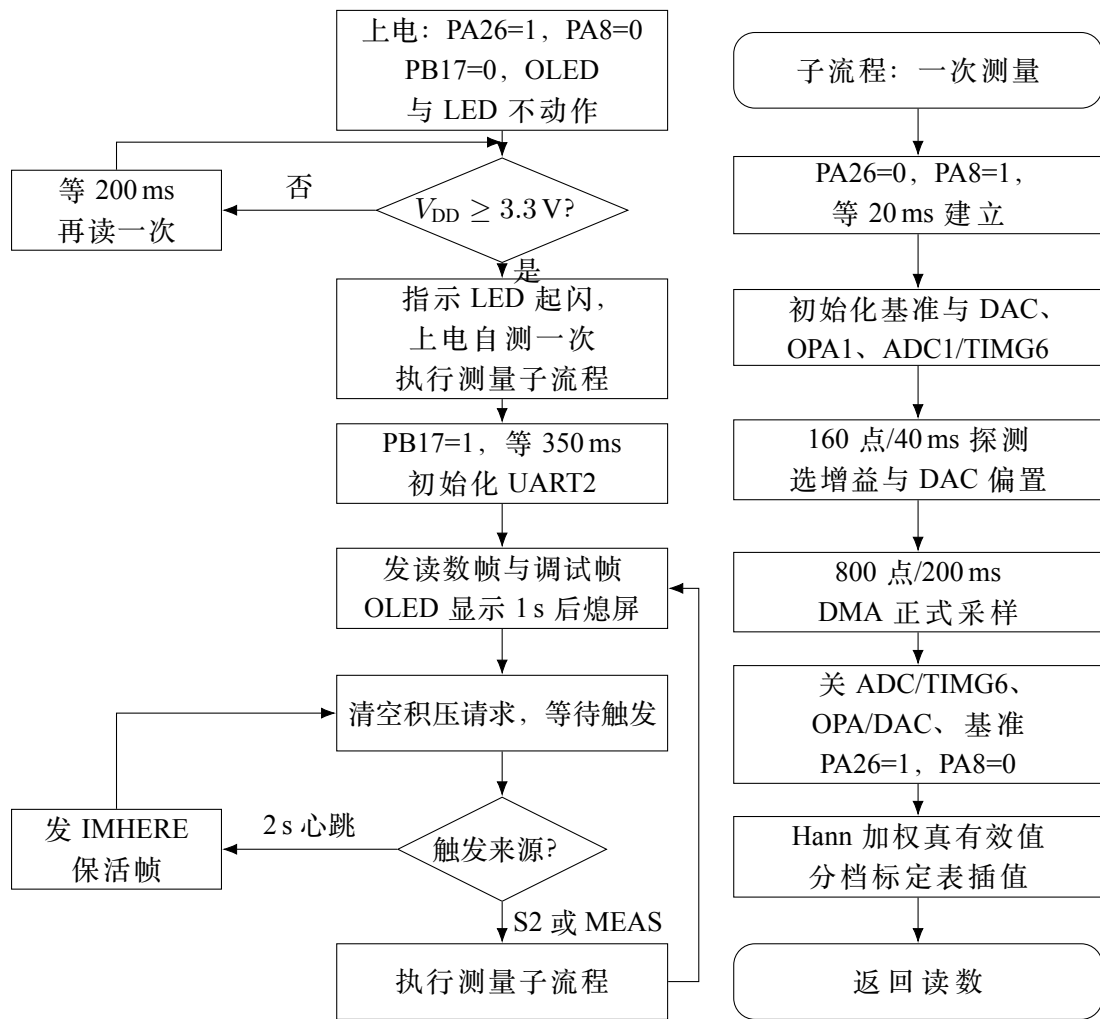


图 2 程序流程图

### 3. 通信协议设计

链路为 HC-42 透传串口，速率 115200 bit/s。透传不提供分包与长度字段，故用面向行的 ASCII 协议，以换行作自同步定界符：接收端在任意位置接入或丢失字节后，都能在下一个换行重新对齐。读数一律由读表器请求产生。

(1) 读表器 → 电流表，请求帧：

MEAS 或 MEAS,ADDR=dd

前者为广播，后者仅编码开关为 dd 的电流表应答。命令带地址，是因为同一台电流表在不同时刻可以被拨成不同地址，仅凭连接无法确定对端身份。

(2) 电流表 → 读表器，读数帧与调试帧：

METER\_TEST,ADDR=dd,CURRENT\_MA=iiii,STATUS=ssss

METER\_CAL,ADDR=dd,RMS=257.83,GAIN=1,FLAG=OK,MEAN=501,PP=856

CURRENT\_MA 为有效值，单位 mA；STATUS 取 NORMAL、LOW、HIGH，

按 0.2 A 与 2.0 A 判定，没有 OFFLINE——离线是读表器的判断，电流表无法报告自己不在。调试帧同次产生，RMS、GAIN 用于建表，MEAN、PP 是探测帧测得的输入直流与峰峰值，FLAG 是该次读数的质量标志。

(3) 电流表 → 读表器，保活帧：

IMHERE,ADDR=dd

空闲时每 2 s 一次，不含读数，也不唤醒模拟电路。拨动编码开关时旧地址的心跳停止、新地址出现，读表器据此更新在场地址；负载断开后电流表失电，心跳消失即为离线判据。请求超时只说明本次未读到，不据此判定离线。

#### 4. 其他重要模块设计

Android 读表器基本模式一对一手动读取；自动模式一键启动，每 120 s 对在场地址依次请求，同一连接上只允许一个未完成请求。读数写入 SQLite，环形保存最近 1000 条，掉电不丢，历史与趋势共用这份数据。低于 0.2 A 与高于 2.0 A 的每一次读数都报警，而不是只在状态跳变时报：读数稀疏，同一状态的相邻两次读数是两个独立事件。

### 四、 测试方案及测试结果分析

#### 1. 测试仪器与测试条件

测试在室温下进行，回路按题目要求接成 36 V 试验回路：隔离变压器 36 V 输出串入标定电流表、取能/传感铁芯初级 ( $N_1 = 5$  匝) 和自制负载。所用仪器与测试条件见表 1。

表 1 测试仪器与测试条件

| 名称    | 型号或规格                            | 用途                 |
|-------|----------------------------------|--------------------|
| 交流电源  | 熙硕 STG-500W                      | 经隔离变压器为试验回路供电      |
| 隔离变压器 | 单相 220 V/36 V, 60 VA             | 提供 36 V 试验回路, 强电隔离 |
| 自制负载  | 视在功率约 50 VA, 0 ~ 2.0 A<br>连续可调   | 设定被测负载电流           |
| 标定电流表 | SIGLENT SDM3055X-E 五<br>位半数字多用表, | 作标准表, 与被测电流表读数比对   |
| 数字示波器 | SIGLENT SDS2102X PLUS            | 观察储能电压建立、传感窗口与首帧时刻 |
| 秒表    | 分辨力 0.01 s                       | 记录从接通负载到正常工作的时间    |
| 无线读表器 | Android 手机一部, 运行自<br>制读表器应用      | 接收读数帧, 读取被测表示值     |
| 环境条件  | 室温, 线路电压 36 V                    | 全部测试项共用            |

## 2. 测试方法

(1) 精度测试: 线路电压 36 V, 在 0.12 ~ 2.14 A 范围内设置 8 个负载电流测点。每点稳定后同时记录标定电流表读数  $I_s$  和自制电流表示值  $I_x$ , 按  $(I_x - I_s)/I_s$  计算相对误差。

(2) 不同负载电流下启动时间测试: 每次测试前按下放电按钮把储能电容残余电量放净, 再合上插座总开关接通负载, 用秒表记录从接通到电流表首次给出正常读数的时间。负载电流依次取 0.30、0.32、0.35、0.38、0.40、0.54、0.60、0.70 A。

## 3. 测试结果

精度比对结果见表 2, 不同负载电流下的启动时间见表 3。

表 2 电流测量精度记录表

|         |       |       |       |       |       |       |       |       |
|---------|-------|-------|-------|-------|-------|-------|-------|-------|
| 标准表读数/A | 0.119 | 0.188 | 0.541 | 0.699 | 1.000 | 1.200 | 1.601 | 2.136 |
| 自制表读数/A | 0.119 | 0.188 | 0.542 | 0.698 | 0.998 | 1.202 | 1.599 | 2.132 |
| 相对误差/%  | 0.00  | 0.00  | +0.18 | -0.14 | -0.20 | +0.17 | -0.12 | -0.19 |

表 3 不同负载电流下的启动时间记录表

|        |           |      |      |      |      |      |      |      |
|--------|-----------|------|------|------|------|------|------|------|
| 负载电流/A | 0.30      | 0.32 | 0.35 | 0.38 | 0.40 | 0.54 | 0.60 | 0.70 |
| 启动时间/s | ><br>1260 | 270  | 170  | 153  | 140  | 89   | 75   | 62   |

## 4. 测试结果分析

表 2 的 8 个测点覆盖 0.119~2.136 A。各点绝对相对误差均不超过 0.20%，平均绝对相对误差约为 0.13%，8 个测点全部落在题目 0.5% 的要求以内。最大偏差出现在 1.000 A 处，为 -0.20%；低端 0.119 A 与 0.188 A 两点的示值与标准表读数一致。误差未呈现贯穿全量程的单向变化，说明实测标定能够较好地补偿互感器、整流压降和采样链路的综合非线性。量程低端的原始有效值码宽最小，读数对标定表最敏感，仍是后续细化标定的重点。

表 3 表明，启动时间随负载电流增大而明显缩短：0.40 A 时为 140 s，0.54 A 时降至 89 s，0.70 A 时进一步缩短至 62 s。向低端走则迅速变陡：0.38 A 为 153 s，0.35 A 为 170 s，0.32 A 已需 270 s，而 0.30 A 下观察超过 21 min 仍未完成启动——0.32 A 与 0.30 A 只差 0.02 A，启动时间却由 4.5 min 变为观测不到终点。该结果与取能功率随线路电流增加而提高的规律一致：真正用于储能的是取能功率减去待机功耗，两者接近平衡时这一差值趋于零，启动时间随之发散，故曲线在低端不是缓慢变长而是陡然发散。在现有硬件与启动策略下，0.32 A 是能在 5 min 内完成冷启动的最低实测电流点；若要继续下探，应优先降低整流与稳压损耗、减小待机功耗，或优化储能容量和启动门限。

## 五、 总结

本系统完成了“无源”交流电流表及无线读表器。带抽头电流互感器分时完成取能与传感，储能电容经低压差 LDO 供电，MSPM0G3507 片内 OPA、DAC 和 ADC 实现自动量程、真有效值计算及标定修正，读数通过 BLE 发送至 Android 端。实测 0.119~2.136 A 范围内最大绝对相对误差为 0.20%，平均为 0.13%，满足题目 0.5% 的要求；0.32~0.70 A 冷启动时间由 270 s 降至 62 s，说明总体方案可行。优点是无需外部电源、外围器件少、功耗较低，并支持本地显示、无线读数、报警和记录；不足是 0.30 A 附近难以冷启动。后续应降低整流与稳压损耗，细化低端标定并优化启动门限。

附录

1. 整流、钳位与储能电路

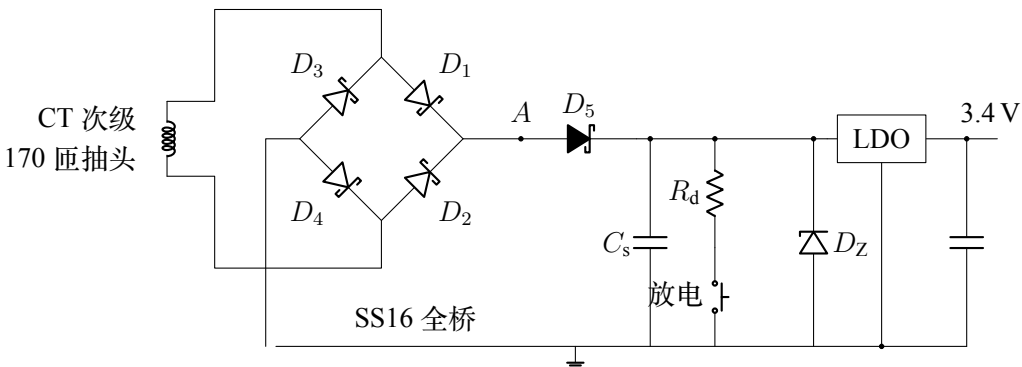


图 1 整流、钳位与储能电路

2. 三种稳压方案

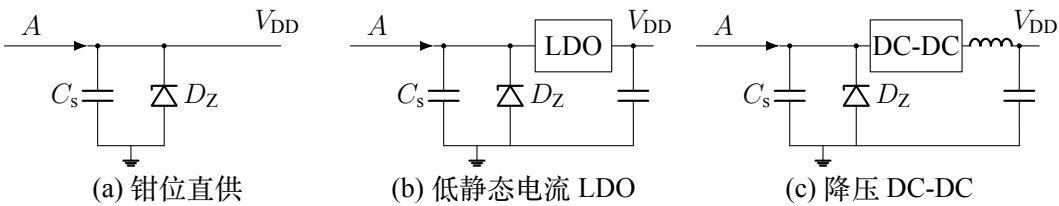


图 2 三种稳压方案

3. 取电与传感分时控制状态

表 1 取电与传感分时控制状态

| 工作状态  | PA26 | PA8 | MOS 状态 | 能量与信号路径              |
|-------|------|-----|--------|----------------------|
| 空闲/取电 | 高    | 低   | 均关断    | 传感前端隔离，互感器输出全部用于整流储能 |
| 测量/传感 | 低    | 高   | 均导通    | 传感前端接入，储能电容独立维持系统供电  |

4. 两段固件代码 (embedded Rust)

固件以 embedded Rust 编写，no\_std 裸机运行，异步执行器用 embassy。

代码 1 把标定表的单调性检查放在编译期。cal\_strictly\_ascending 是 const fn，编译器在编译期遍历整张表，出现一对逆序或转置的 (LSB, 安培) 时构建失败，并指明是哪张表。判据写成正条件而不是取反：与 NaN 的比较一律为假，写错的浮点字面量因此落进 else，不必另设一条检查。

C 的 `_Static_assert` 要求操作数为整型常量表达式，既不能循环遍历一张表，也不能比较 `float`，同样的检查只能移到运行时；而那时表已经烧进片子，插值走错分段给出的是量纲正确、数值错误的读数。

代码 1 `src/cal.rs`: 标定单调性的编译期校验

```
1 /// Both columns must increase together, checked at
2 /// compile time: the table is typed in by hand.
3 pub const fn cal_strictly_ascending(
4     cal: &[(f32, f32)],
5 ) -> bool {
6     let mut i = 1;
7     while i < cal.len() {
8         // Positive condition, not a negated one:
9         // any comparison against NaN is false, so
10        // a NaN falls to the `else` as well.
11        if cal[i].0 > cal[i - 1].0
12            && cal[i].1 > cal[i - 1].1
13        {
14            i += 1;
15        } else {
16            return false;
17        }
18    }
19    true
20 }
21
22 // The 68-row table is walked at compile time.
23 const _: () = assert!(
24     cal_strictly_ascending(CAL_X1),
25     "{}",
26     CAL_ORDER_MSG,
27 );
```

代码 2 是主循环对两路触发的等待。`select` 同时挂起按键下降沿与蓝牙命令两个 `future`，先到者返回，另一个在原地被丢弃：按键那侧丢掉的是尚未处理的边沿，蓝牙那侧留下的缓冲字节由随后的 `discard_pending` 清除。两路在语义上是同一个请求，因此不区分是谁触发的。等待期间执行器空转，模拟部分全部掉电。

未完成的 `future` 被丢弃时其析构函数运行，注册的中断与持有的缓冲随之释放，不需要单独写取消路径。C 里等价的实现一般是中断标志加状态机，两路触发各自的清理要手写。

代码 2 src/main.rs: 按键与蓝牙命令的两路等待

```
1 /// Idle until something asks for a reading,
2 /// announcing this meter's address every
3 /// `HEARTBEAT_MS` in the meantime.
4 async fn wait_for_request(
5     trigger: &mut Input<'static>,
6     link: &mut Link,
7     address: &Address,
8 ) {
9     // Whichever arrives first wins and the other
10    // future is dropped where it stands. Which one
11    // won does not matter: the button and a `MEAS`
12    // command are the same request.
13    select(
14        trigger.wait_for_falling_edge(),
15        link.wait_command(address),
16    )
17    .await;
18 }
```

## 5. 电流表硬件原理图

## 6. 模拟前端原理图

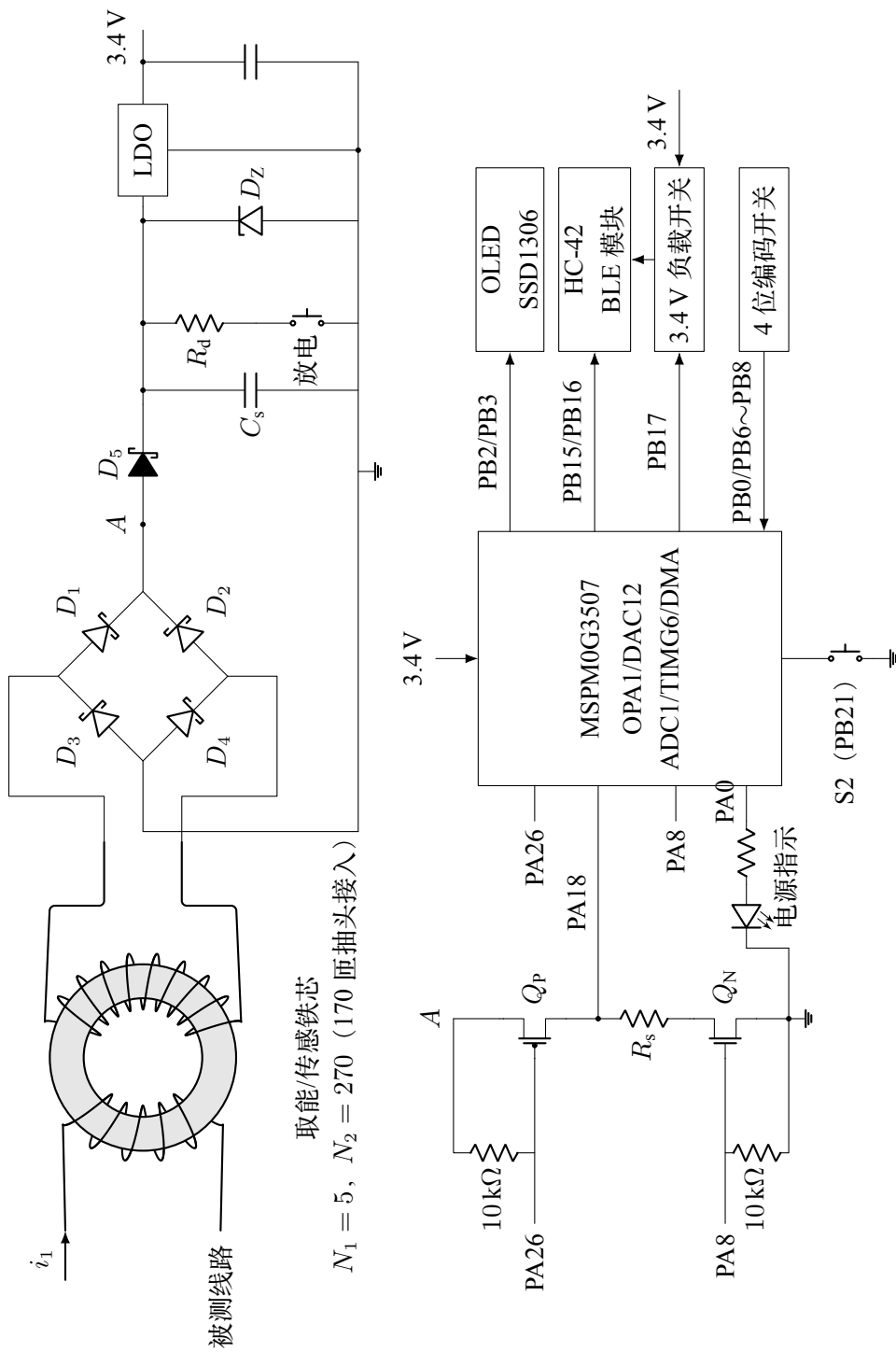


图 3 电流表硬件原理图

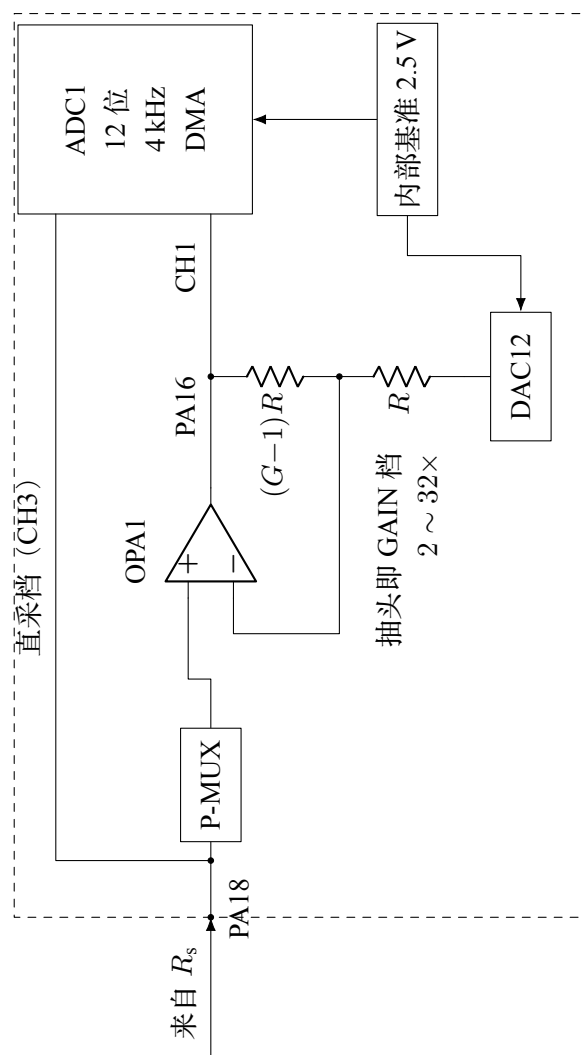


图 4 模拟前端原理图